

UT-04: ESTRUCTURAS DE DATOS. ARRAYS Y CADENAS DE CARACTERES

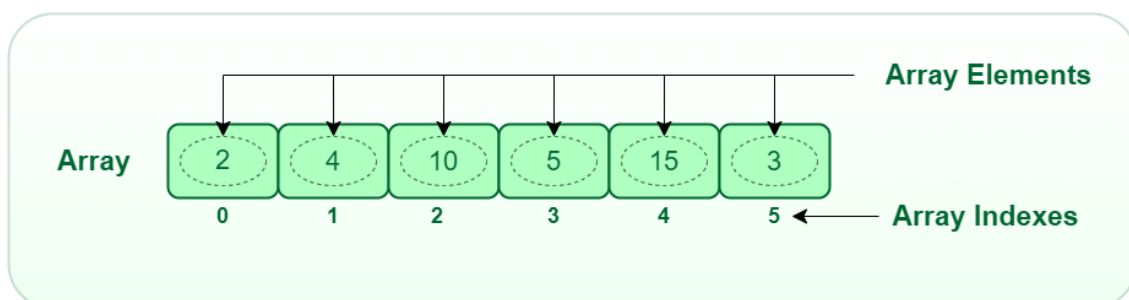
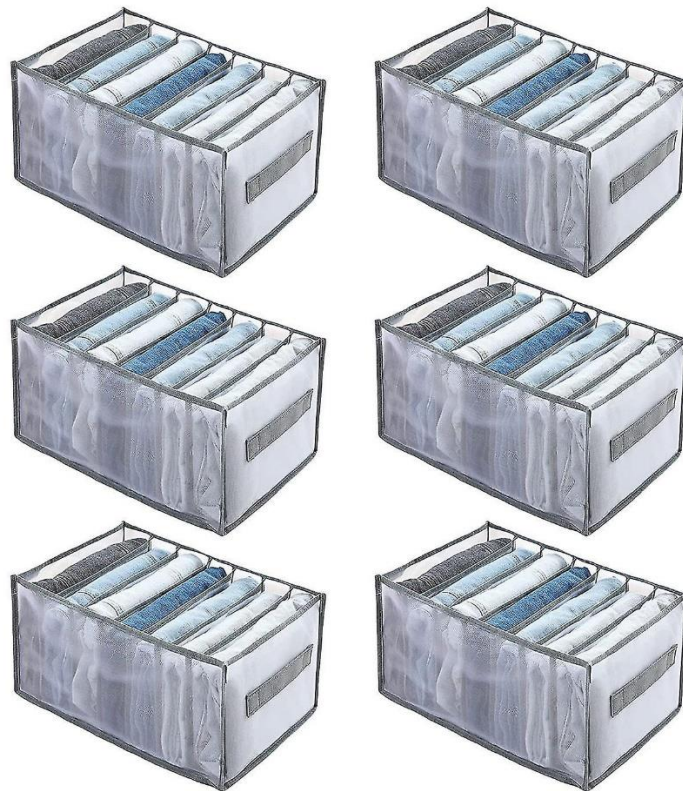
¿Qué vamos a ver?

- Arrays unidimensionales
 - Declarar y crear un array
 - Inicializar un array
 - Acceder a los elementos de un array
- Operaciones
 - Recorrido
 - Copia
 - Ordenación
 - Comparación
 - Búsqueda
 - Inserción de elementos
 - Eliminación de elementos
 - La clase Arrays
- Arrays bidimensionales o matrices
 - Declarar y crear una matriz
 - Inicializar una matriz
 - Acceder a los elementos de una matriz
- Cadenas de caracteres. String
 - Métodos de la clase string
 - Formato de cadenas en java
 - Operaciones
 - Acceso a elementos
 - Conversiones
 - Concatenaciones
 - Expresiones regulares

1. ARRAYS UNIDIMENSIONALES

Los arrays unidimensionales son tipos de datos lineales, compuestos, ordenados y homogéneos, es decir, que una variable de tipos array almacena varios valores, no solo uno, como habíamos visto hasta el momento y todos son del mismo tipo (int, float, char, String, ...). Además, cada uno de estos valores ocupan una determinada posición dentro del array, identificada por un índice y al ser lineales cada elemento tiene como máximo otro elemento que le preceda y otro que le siga.

Con una imagen lo vemos de una forma más sencilla:



1.1. Declaración e inicialización

En java podemos declarar e inicializar los arrays unidimensionales de dos maneras:

1. Solo declarar un array vacío de la longitud que deseamos

```
int edades[] = new int[3];  
int[] edades = new int[3];
```

En este caso inicializaremos los valores más adelante. Para ello, haremos uso de los índices que apunta a cada una de las posiciones del array. Algo MUY IMPORTANTE que debemos tener en cuenta es que los índices de LOS ARRAYS SIEMPRE EMPIEZAN A CONTAR DESDE 0, de tal forma que el índice del último elemento será el de la longitud del arrays menos 1.

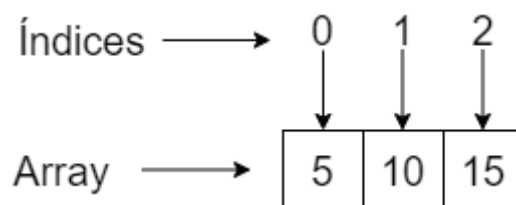
Para inicializar el valor a cada una de las posiciones del array haremos lo siguiente:

```
int edades[];  
edades[0] = 5;  
edades[1] = 10;  
edades[2] = 15;
```

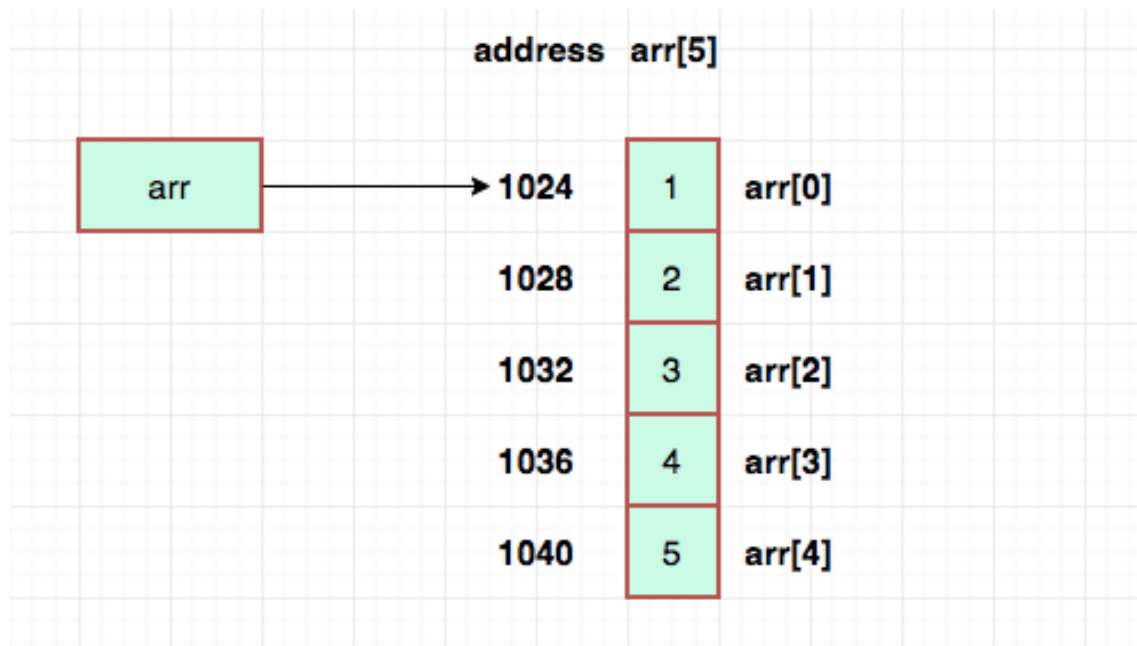
2. Declarar un array e inicializando su contenido directamente y de esta forma le damos de una forma implícita el tamaño que deseamos.

```
int edades[] = {5, 10, 15}  
int[] edades = {5, 10, 15};
```

Ambos casos se pueden representar de la siguiente manera:



Cuando nosotros creamos una array, ya sea por medio de la instrucción `new` o inicializando directamente, nuestra variable lo que almacena es una referencia, o puntero, a la dirección donde se almacena primera posición del array en la memoria:



How we perceive an array:

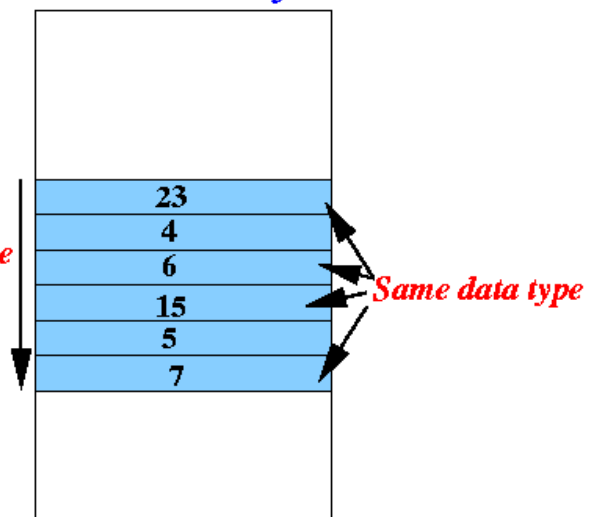
Array:

23	4	6	15	5	7
0	1	2	3	4	5

*Consecutive
memory
locations*

How it is stored in memory

RAM memory



1.2. Acceder a los elementos de un array

Para acceder individualmente a cada elemento del array solo debemos utilizar su índice, de tal forma que escribimos el nombre del array seguido del índice al que queremos acceder, poniéndolo entre corchetes (nombreArray[índice]).

Retomando el ejemplo anterior imaginemos que queremos almacenar el contenido de las tres posiciones del array en tres variables distintas. Haríamos lo siguiente:

```
int edadPosicion0 = edades[0];  
int edadPosicion1 = edades[1];  
int edadPosicion2 = edades[2];
```

De tal forma que:

- edadPosicion1 tendrá ahora el valor 5
- edadPosicion2 tendrá ahora el valor 10
- edadPosicion3 tendrá ahora el valor 15

Este modo de acceso se puede utilizar tanto para conocer el valor del elemento indicado, como para cambiar su valor:

```
numeros[1] = 20;
```

Para cualquier array unidimensional el índice del primer elemento será 0.

```
int edadPrimero = edades[0];
```

Además, para cualquier array unidimensional el índice del último elemento será el de la longitud del array menos 1.

En el caso de no conocer la longitud del array, podemos utilizar la siguiente instrucción:

```
nombreArray.length;
```

De tal forma que si queremos acceder al último elemento de un array podremos hacer lo siguiente:

```
int edadUltimo = edades[nombreArray.length - 1];
```

Que en este caso sería lo mismo que:

```
int edadUltimo = edades[2];
```

Ya que la longitud del array es 3, el último índice sería $3 - 1 = 2$.

Debemos tener cuidado a la hora de acceder a las posiciones de los arrays, puesto que, si accedemos a un índice que no exista, ya sea porque es menor que 0 o mayor o igual a la longitud del array nos dará un error que finalizará la aplicación. En el ejemplo anterior, no debemos intentar accesos como los siguientes:

```
int edadPosicion0 = edades[-1];  
int edadPosicion1 = edades[3];  
int edadPosicion2 = edades[20];
```

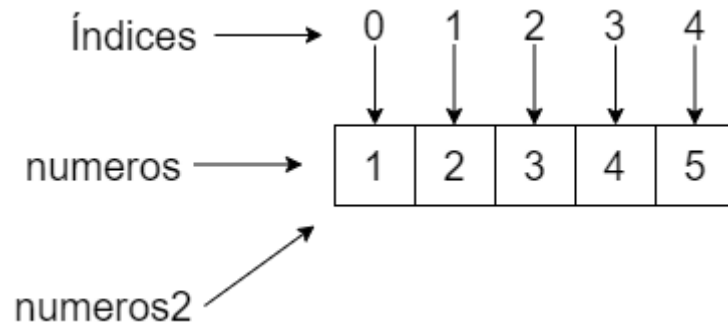
Ya que en cualquiera de estos casos nos dará error.

1.3. Referencia a un array, null y recolector de basura

Tenemos la posibilidad de que varias variables hagan referencia a un mismo array, para ellos solo tenemos que asignar dicho array a una variable de tipo array del mismo tipo que le original.

```
int numeros[] = {1, 2, 3, 4, 5};  
  
int numeros2[];  
  
numeros2 = numeros;
```

Esto se representaría gráficamente de la siguiente manera:



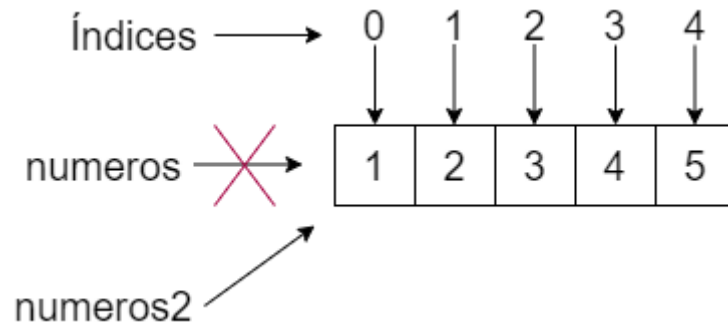
De esta forma ahora podremos acceder a la misma parcela de memoria desde las variables `numeros` y `numeros2`. Por ejemplo, ahora mismos `numeros[4]` y `numeros2[4]` estarían haciendo referencia al mismo valor, es decir, 5.

Debemos tener en cuenta que cualquier modificación que hagamos en el contenido del array desde cualquiera de las dos variables también afectará el otro, puesto que están apuntando a la misma posición de memoria, es decir, a los mismos datos.

En el caso de que queramos que una variable array deje de hacer referencia una parcela de memoria, debemos utilizar el valor `null`, y dicha referencia desaparecerá.

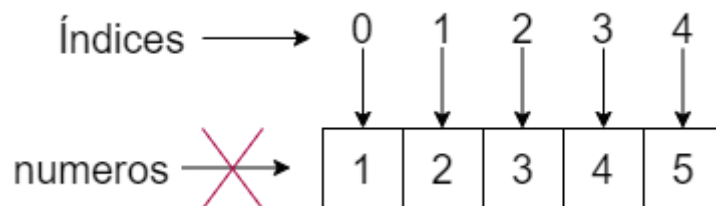
```
int numeros[] = {1, 2, 3, 4, 5};  
  
int numeros2[];  
  
numeros2 = numeros;  
  
numeros = null;
```

Continuando con el ejemplo anterior si después de copiar la referencia de `numeros` en `numeros2` asignásemos el valor `null` a `numeros`, este ya no podría acceder al array, puesto que habría desaparecido la referencia que almacenaba.



Es decir, podríamos seguir accediendo a la región de memoria desde `numeros2`, pero ya no tendríamos acceso desde `numeros`.

En el ejemplo que estamos viendo, los datos se mantienen puesto que sigue habiendo una variable que hace referencia a ellos, pero... ¿Qué ocurriría si a la única variable que tiene acceso a esa zona de la memoria le asignamos el valor `null`?



Pues que esa región de memoria ya sería inaccesible y por tanto no puede utilizarse. En esta situación entraría en marcha lo que denomina “Recolector de basura”, que es el proceso encargado de limpiar la memoria de datos que no pueden utilizarse, para que ese espacio pueda ser reutilizado.

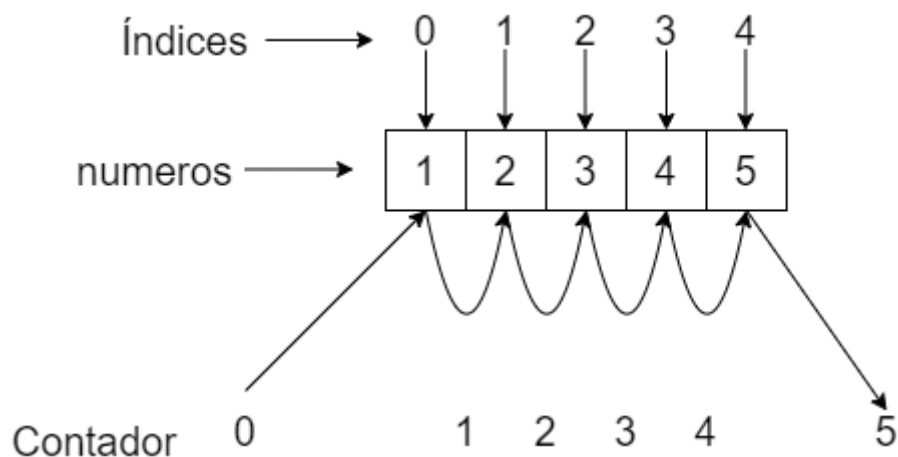
2. RECORRIDO DE ARRAYS

Recorrer el contenido de los arrays es muy sencillo utilizando los bucles, en especial el bucle `for`. Vamos a ver cómo recorrer un array unidimensional con cualquiera de ellos:


```
public static void main(String[] args) {  
    int numeros[] = {1, 2, 3, 4, 5};  
  
    //Recorrido con for  
    for(int indice=0; indice<numeros.length; indice++){  
        System.out.print(numeros[indice] + " ");  
    }  
    System.out.println();  
  
    //Recorrido con while  
    int indice = 0;  
    while (indice < numeros.length){  
        System.out.print(numeros[indice] + " ");  
        indice++;  
    }  
    System.out.println();  
  
    //Recorrido con do while  
    indice = 0;  
    do{  
        System.out.print(numeros[indice] + " ");  
        indice++;  
    } while (indice < numeros.length);  
    System.out.println();  
}
```

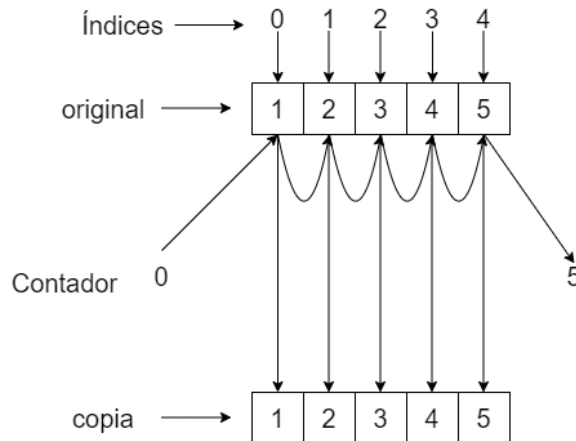
Esto daría como resultado por consola:

```
1 2 3 4 5  
1 2 3 4 5  
1 2 3 4 5
```



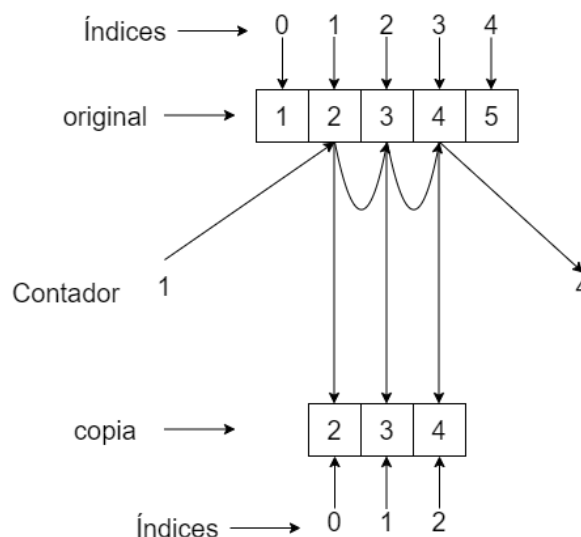
3. COPIA DE ARRAYS UNIDIMENSIONALES

Para realizar la copia completa de un array únicamente tendremos que crear un nuevo array con el mismo tamaño que el que original, e ir copiando los distintos elementos mientras que lo vamos recorriendo.



También podremos realizar copias parciales de un array marcando el elemento inicial y final entre los que queremos copiar.

Por supuesto, podremos realizar tantas variaciones como nos permita la imaginación.



Por supuesto, podremos realizar tantas variaciones como nos permita la imaginación.

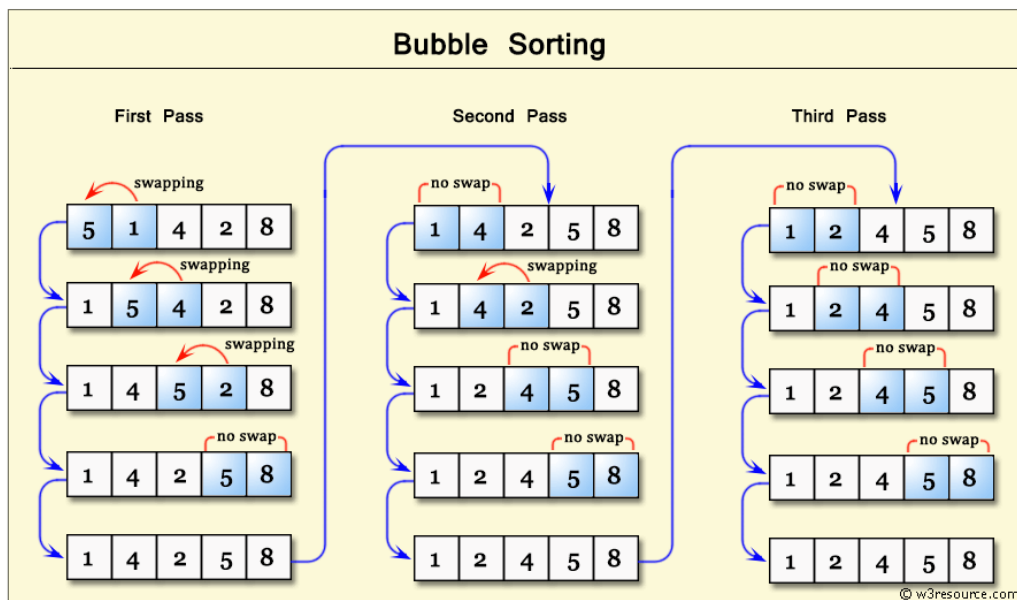
4. ORDENACIÓN DE ARRAYS UNIDIMENSIONALES

Existen una gran cantidad de algoritmos mediante los que podemos ordenar el contenido de un array unidimensional. La gran mayoría están orientados a arrays que contienen valores numéricos.

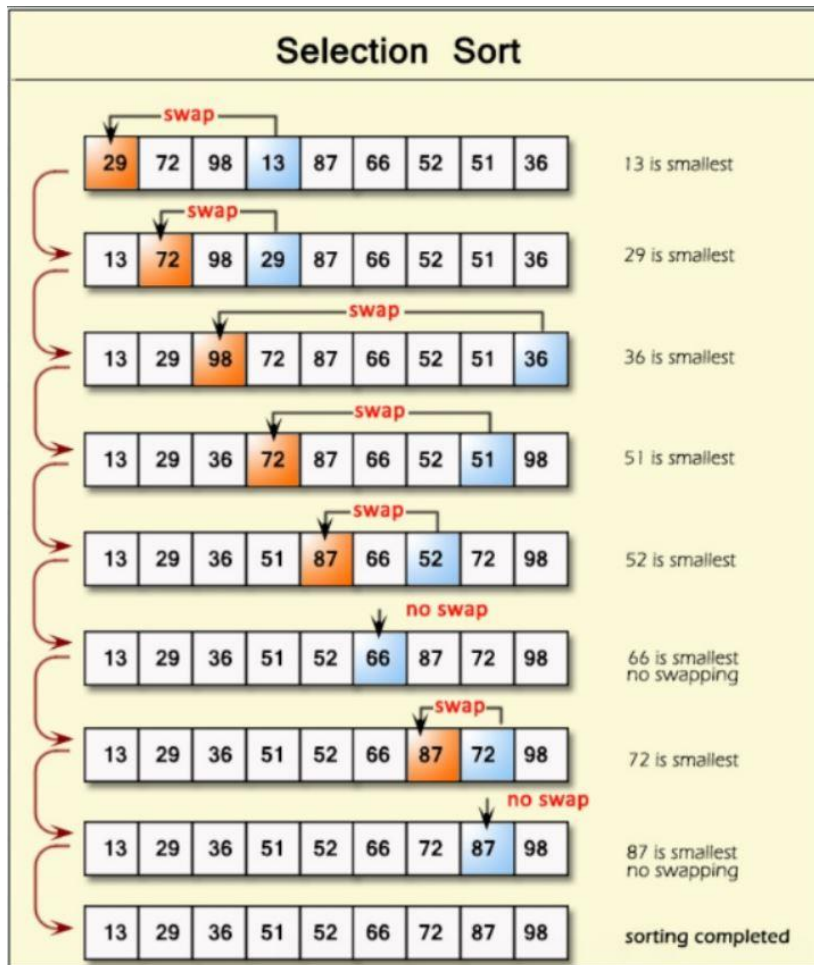
En esta unidad vamos a centrarnos en tres principales. Primero veremos cómo funcionan en una serie de vídeos y después veremos que se pueden implementar muy fácilmente.

Estos tres algoritmos de ordenación son:

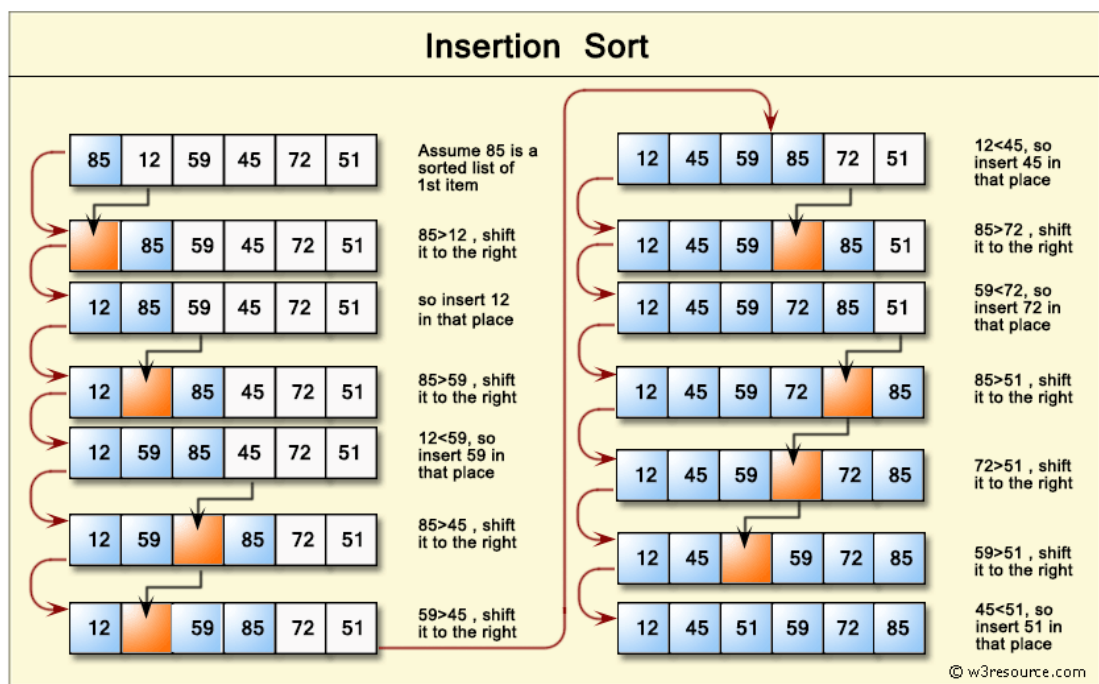
- Algoritmo de la burbuja



- Algoritmo de selección



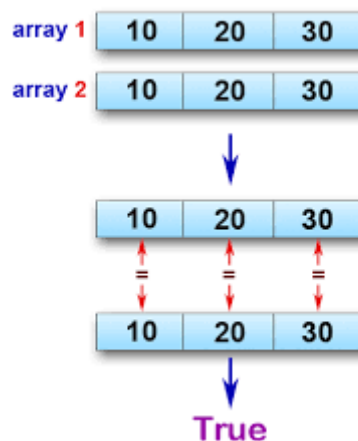
- Algoritmo de inserción



5. COMPARACIÓN DE ARRAYS UNIDIMENSIONALES

Para conocer si dos arrays son iguales, seguiremos los siguientes pasos:

1. Comprobar que tienen la misma longitud. Si tienen distinta longitud son distintos.
2. Si tienen la misma longitud, ordenamos los arrays.
3. Recorremos ambos arrays a la vez y vamos comparando el contenido de cada una de las posiciones con el mismo índice. En el caso de que los valores de la misma posición de los dos arrays sean distintos, los arrays son distintos.
4. Si ninguno de los casos anteriores nos ha dado como resultado que los arrays son distintos, los arrays son iguales.



6. BÚSQUEDA DE ELEMENTOS ARRAYS UNIDIMENSIONALES

Para buscar de forma elementos en arrays unidimensionales tenemos dos opciones:

1. Si el array no está ordenador lo recorreremos de forma secuencial desde el principio hasta el final. De tal forma que si lo encontramos interrumpimos la búsqueda y si no devolveremos que el elemento no se encuentra en el array.

Search Operation in Unsorted Array



2. En el caso de que el array sea ordenado tenemos posibilidades:

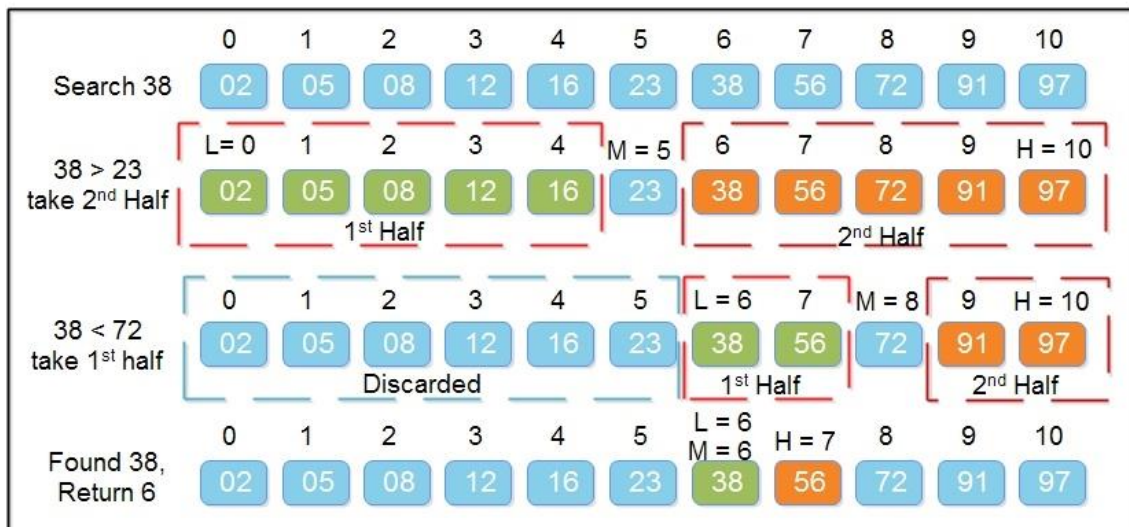
- a. **Secuencial:** También lo recorreremos de principio a fin, pero en este caso interrumpiremos la búsqueda si encontramos el valor o si encontramos uno más alto que él. En el primer caso habríamos encontrado el elemento que buscábamos, y en el segundo sabríamos que no existe en el array.

Linear Search



- b. **Binaria:** Algoritmo mediante el cual vamos dividiendo el array en dos partes y nos quedamos con la parte en donde puede estar ese valor.

Binary Search



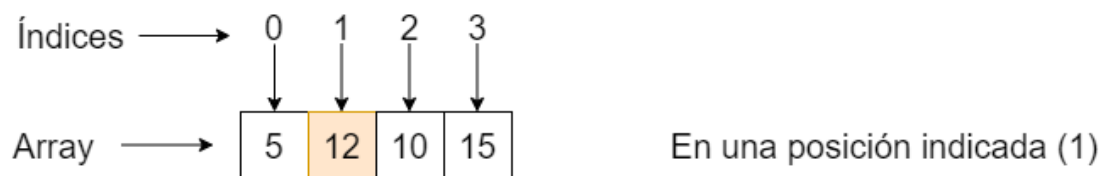
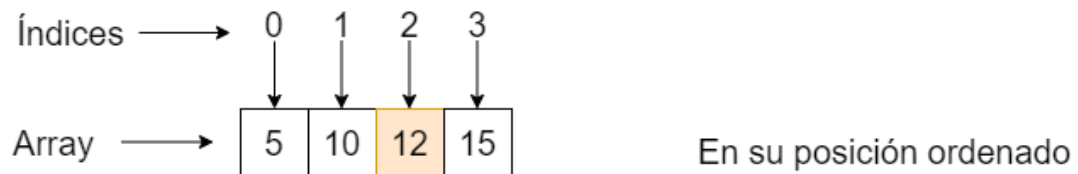
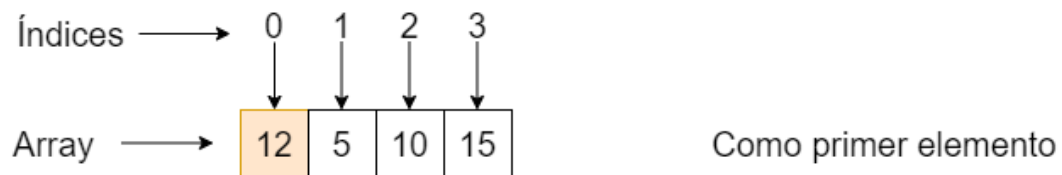
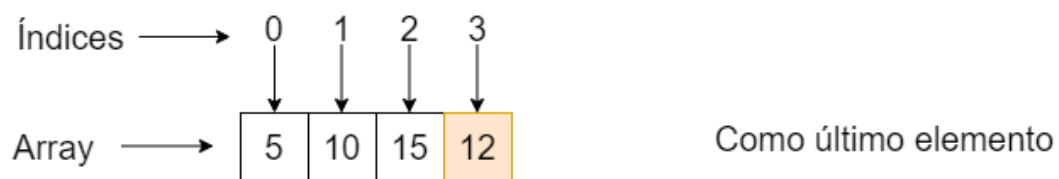
7. INSERCIÓN DE ELEMENTOS EN UN ARRAY

A la hora de insertar un nuevo elemento en un array disponemos de varias posibilidades, pero en todos los casos necesitaremos de un nuevo array con una longitud igual a la longitud del array original + 1.

Las distintas opciones son las siguientes:

1. Insertar un elemento al final de un array.
2. Insertar un elemento al inicio de un array.
3. Inserta un elemento donde le corresponde en un array ordenado.
4. Insertar un elemento en una posición determinada.

TIPOS DE INSERCIÓN



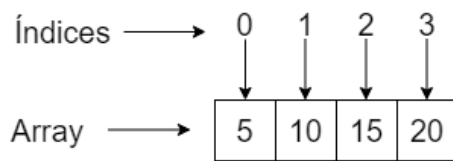
8. ELIMINACIÓN DE ELEMENTOS EN UN ARRAY

A la hora de insertar un nuevo elemento en un array disponemos de varias posibilidades, pero en todos los casos necesitaremos de un nuevo array con una longitud igual a la longitud del array original - 1.

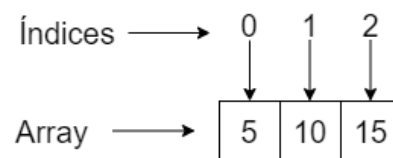
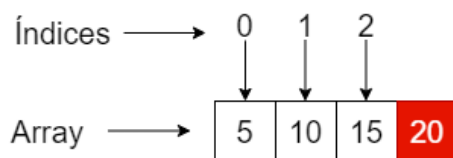
Las distintas opciones son las siguientes:

1. Eliminar un elemento al final de un array.
2. Eliminar un elemento al inicio de un array.
3. Eliminar un elemento determinado dentro de un array, ordenado o no ordenado.
4. Eliminar un elemento en una posición determinada.

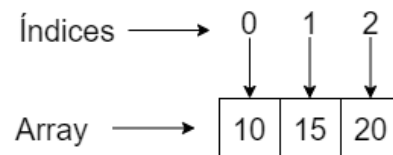
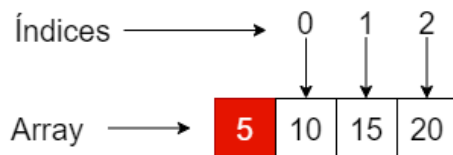
TIPOS DE ELIMINACIÓN



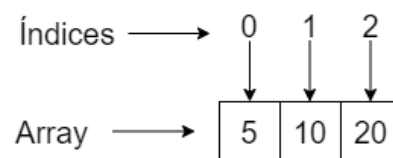
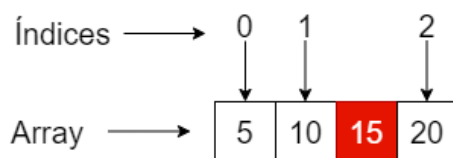
Como último elemento



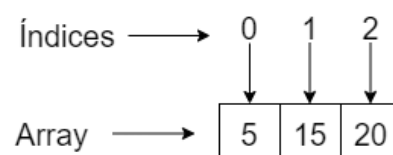
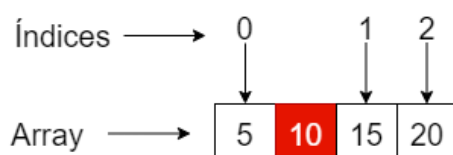
Como primer elemento



Valor indicado (15)



En una posición indicada (1)



9. LA CLASE ARRAYS

Después de realizar de forma “manual” cada una de las operaciones que hemos visto: copia, comparación, búsqueda, inserción y eliminación de arrays unidimensionales, debemos saber que existe una clase (Arrays) que nos proporciona estas operaciones de una forma “automática” a través de métodos. Para poder utilizar esta clase debemos añadir la siguiente importación a nuestros programas:

```
import java.util.Arrays;
```

A continuación, veamos en qué nos puede ayudar la clase Arrays:

9.1. Copia de arrays

La clase Arrays nos proporciona dos métodos mediante los cuales podemos realizar copias de arrays:

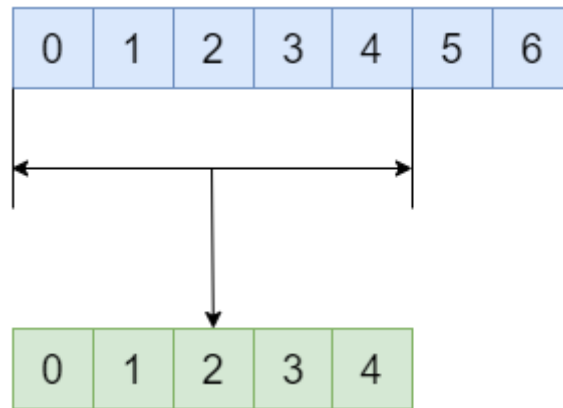
- El método **Arrays.copyOf(array, nuevaLongitud)** copia un array construyendo uno nuevo que se ajustará a la nueva longitud indicada. Veamos tres ejemplos:

```
int numeros[] = {1, 2, 3, 4, 5};

int[] copia1 = Arrays.copyOf(numeros, numeros.length);
// devolverá {1,2,3,4,5}

int[] copia2 = Arrays.copyOf(numeros, 3);
// devolverá {1,2,3}

int[] copia3 = Arrays.copyOf(numeros, 7);
// devolverá {1,2,3,4,5,0,0}
```



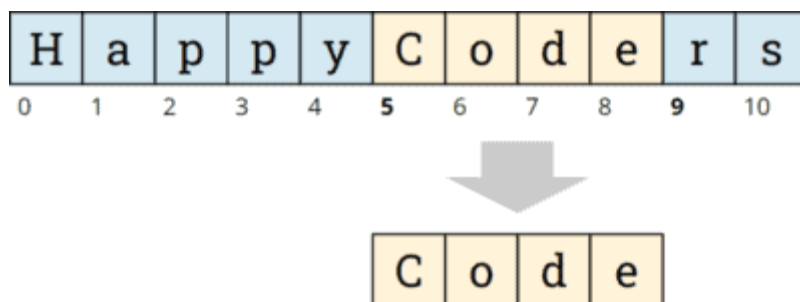
- El método **Arrays.copyOfRange(array, posIni, posFin)** realiza una copia parcial del array, empezando por la posición de inicio y finalizando en la posición anterior a fin. En este caso, al utilizar posiciones, debemos tener cuidado de movernos en los rangos del propio array original.

Veamos dos ejemplos:

```
int numeros[] = {1, 2, 3, 4, 5};

int[] copia1 = Arrays.copyOfRange(numeros, 0, numeros.length);
//devolverá {1,2,3,4,5}

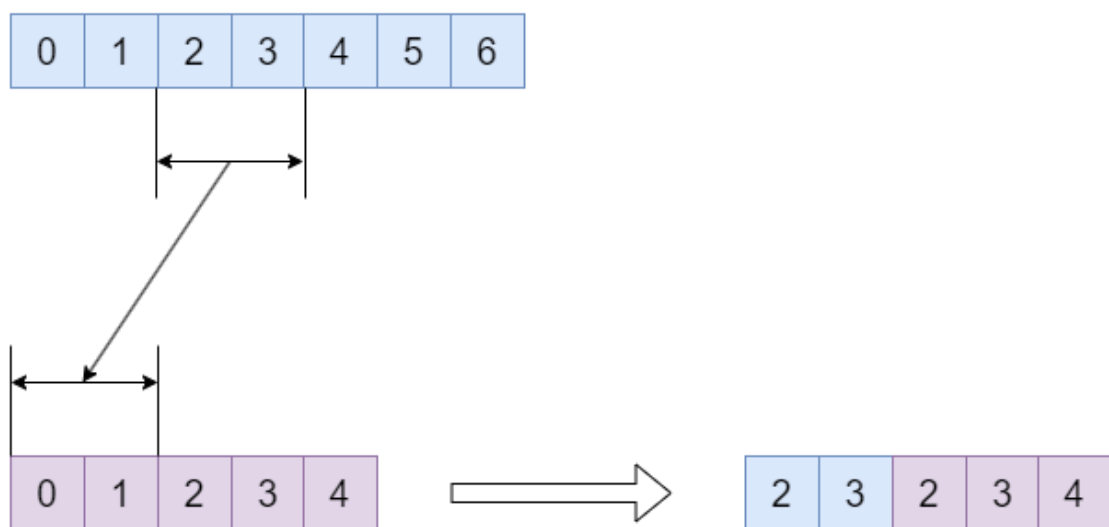
int[] copia2 = Arrays.copyOfRange(numeros, 1, 4);
//devolverá {2,3,4}
```



Además de estos dos métodos de la clase **Arrays** contamos con un tercer método, pero en este caso de la clase **System**, que tiene el nombre de **System.arraycopy(arrayOrigen, posOrig, arrayDestino, posDest, longitud)**. Con este método realizaremos copias en el arrayDestino, a partir de la posición de destino, los datos del arrayOrigen, comenzando en la posición origen. El

parámetro longitud determina el número de elementos que se copiarán entre los dos arrays. Veamos algún ejemplo:

```
int arr1[] = {0, 1, 2, 3, 4, 5};  
int arr2[] = {5, 10, 20, 30, 40, 50};  
  
System.arraycopy(arr1, 0, arr2, 0, 1);  
// arr2 pasará a ser {0, 10, 20, 30, 40, 50}  
  
System.arraycopy(arr1, 1, arr2, 3, 2);  
// arr2 pasará a ser {5, 10, 20, 1, 2, 50}
```



9.2. Comparación de arrays

Para comparar dos arrays podemos utilizar el método **Arrays.equals(array1, array2)**, la cual nos devolverá un booleano true si los dos arrays son iguales, y false si no lo son. Pero debemos tener en cuenta que solo tomará dos arrays como iguales si tiene los mismos datos y en el mismo orden.

```
if(Arrays.equals(arrayNumeros1, arrayNumeros2)){  
    System.out.println("Son iguales");  
}  
else{  
    System.out.println("No son iguales");  
}
```

9.3. Búsqueda en un array

También podemos buscar la primera aparición de un valor dentro de un array utilizando el método **Arrays.binarySearch(array1, valor)**. Para poder realizar esta búsqueda binaria, al igual que en el método “manual”, el array debe estar ordenado. Este método nos devolverá la posición en la que se encuentra el elemento que buscamos, o no devolverá un valor negativo en caso contrario.

```
int pos = Arrays.binarySearch(numeros, numero);
if(pos >= 0){
    System.out.println("Está en el array");
}
else{
    System.out.println("No está en el array");
}
```

9.4. Rellenar array

Para inicializar los valores de un array de una forma cómoda podemos utilizar el método **Arrays.fill(array, valor)**. De esta forma todos los elementos del array tomarán el valor que se ha proporcionado.

```
int arr1[] = {0, 1, 2, 3, 4, 5};

Arrays.fill(arr1, 6);
// arr1 ahora será {6,6,6,6,6,6}
```

9.5. Convertir un array en un String

También tenemos la posibilidad de convertir el contenido de un array en un String, de tal forma que nos sea más sencillo escribirlo pantalla. Para ello debemos utilizar el método **Arrays.toString(array)**.

```
int arr1[] = {0, 1, 2, 3, 4, 5};

System.out.println(Arrays.toString(arr1));
//la salida por pantalla será "[0, 1, 2, 3, 4, 5]"
```

9.6. Ordenación de un array

Para ordenar un array con la clase Arrays, solo necesitaremos usar el método Arrays.sort(array).

```
int[] numeros = {2, 5, 3, 56, 8, 40, 32};  
Arrays.sort(numeros);  
//Ahora numeros será {2, 3, 5, 8, 32, 40, 56}
```

9.7. Inserción en un array

Para insertar elementos en un array no tenemos métodos específicos, pero podemos hacer uso de los métodos de copia para llevarlos a cabo.

9.8. Eliminación de elementos en un array

Para la eliminación de elementos en un array, al igual que ocurre con las eliminaciones, no disponemos de métodos específicos, pero podremos llevarlas a cabo utilizando los métodos de copia.

10. ARRAYS BIDIMENSIONALES O MATRICES

Dentro de cada posición de un array no solo podemos almacenar elemento de tipos primitivos o Strings, sino que podemos almacenar cualquier tipo de objetos, entre ellos otros arrays, de esta forma obtenemos lo que se denominan arrays multidimensionales, lo más conocidos son los arrays bidimensionales, también conocidos como matrices o tablas, y que será en los que nos centraremos en los siguientes apartados...

	0	1	2		n-1
0	a[0][0]	a[0][1]	a[0][2]		a[0][n-1]
1	a[1][0]	a[1][1]	a[1][2]		a[1][n-1]
2	a[2][0]	a[2][1]	a[2][2]		a[2][n-1]
3	a[3][0]	a[3][1]	a[3][2]		a[3][n-1]
4	a[4][0]	a[4][1]	a[4][2]		a[4][n-1]
.	.	.	.		.
.	.	.	.		.
n-1	a[n-1][0]	a[n-1][1]	a[n-1][2]		a[n-1][n-1]

a[n][n]

MI ZONA DE BATALLA

	A	B	C	D	E	F	G	H	I	J
1										
2										
3										
4										
5										
6										
7										
8										
9										
10										

BUQUE DE GUERRA



SUBMARINO



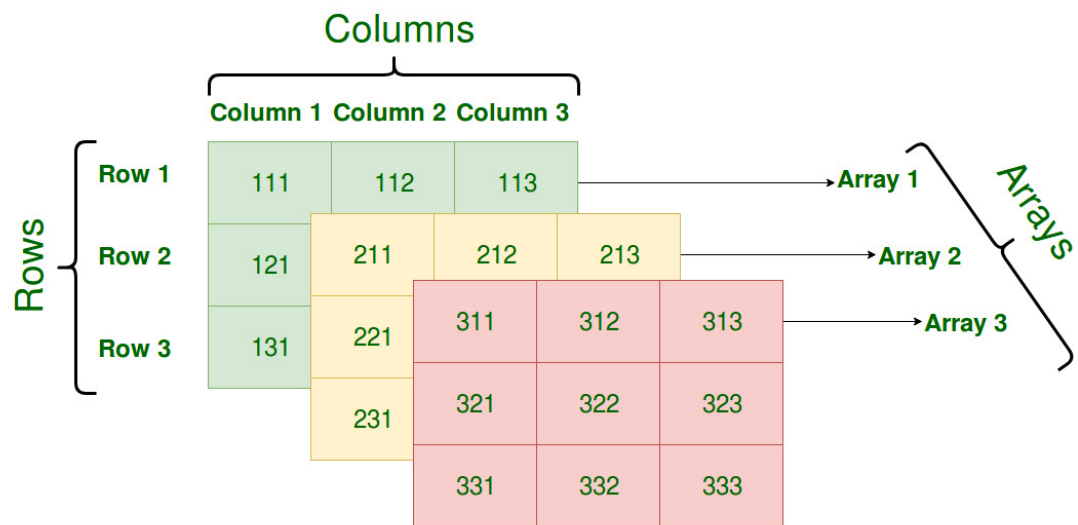
CRUCERO



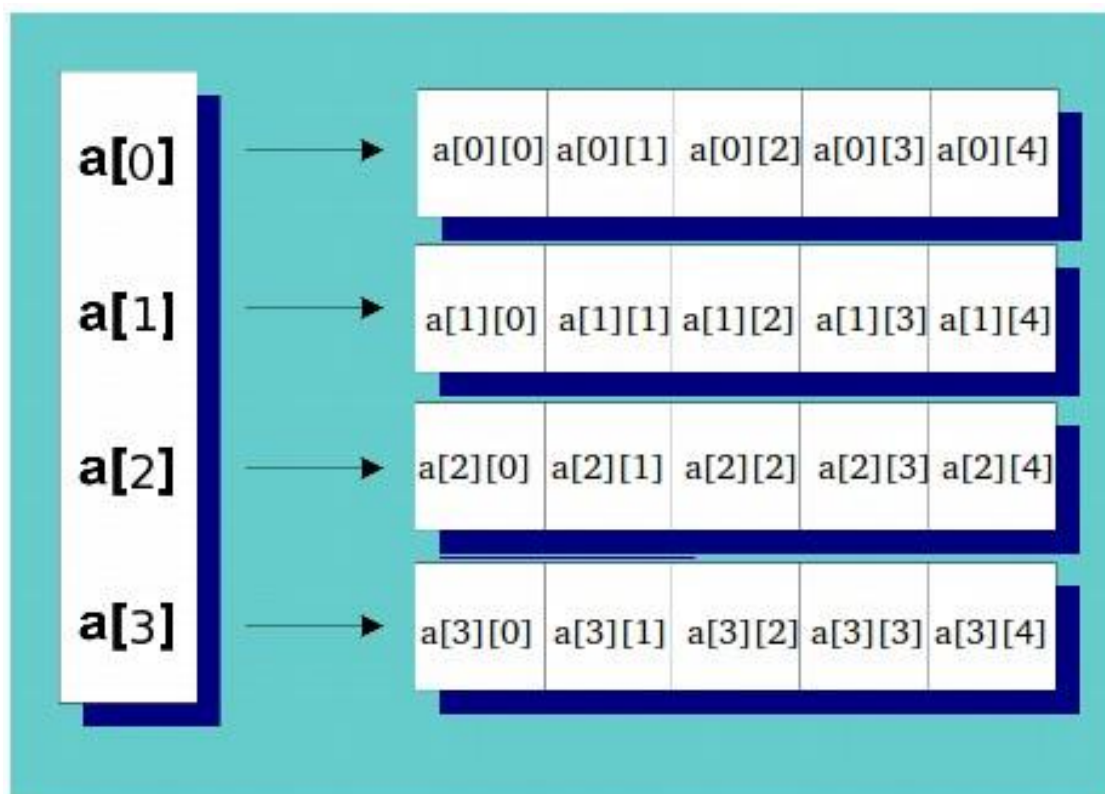
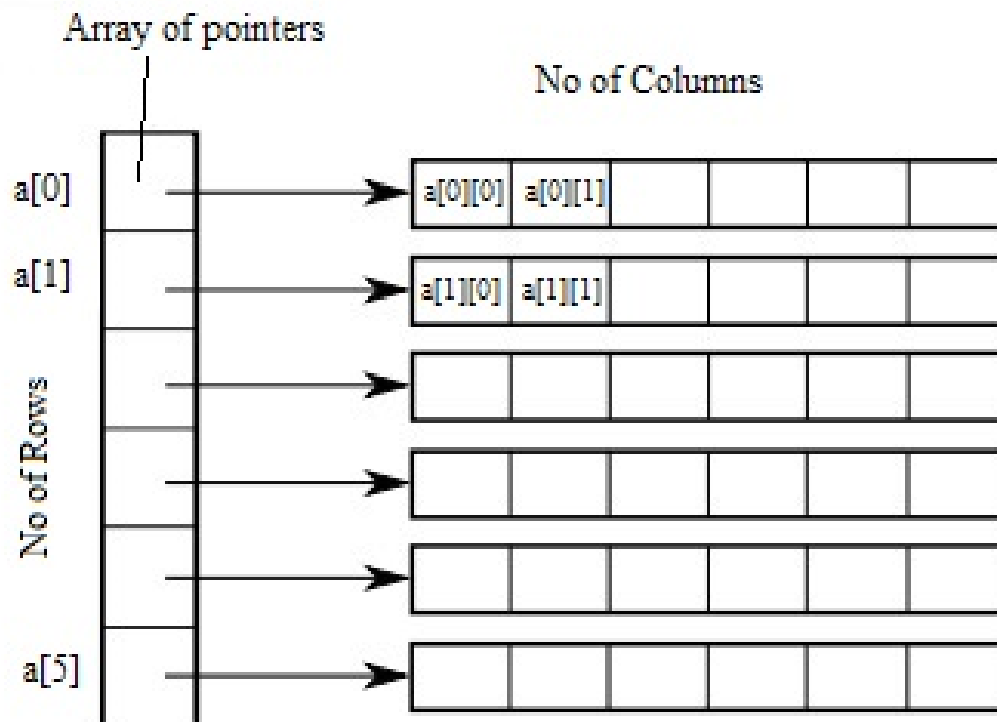
DESTRUCTOR



O los arrays de tres dimensiones...



10.1. Declaración, creación e inicialización de matrices



		Column 1	Column 2	Column 3	Column 4		
Row indexes		0	1	2	3	Column indexes	
Row 1	0	a[0][0] 15	a[0][1] 20	a[0][2] 25	a[0][3] 30	← Array a[0]	
Row 2	1	a[1][0] 20	a[1][1] 30	a[1][2] 40	a[1][3] 50	← Array a[1]	
Row 3	2	a[2][0] 60	a[2][1] 65	a[2][2] 70	a[2][3] 80	← Array a[2]	

A la hora de declarar un array multidimensional lo haremos de forma similar que a los unidimensionales, con la única diferencia de añadir una pareja “[]” por cada dimensión. Ejemplo de matriz bidimensional:

```
int[][] matriz;
```

Para crear las matrices bidimensionales también tenemos tres formas distintas:

Construcción tras haberlo declarado:

```
matriz = new int[3][4];
```

Declaración y construcción a la vez:

```
int[][] matriz = new int[3][4];
```

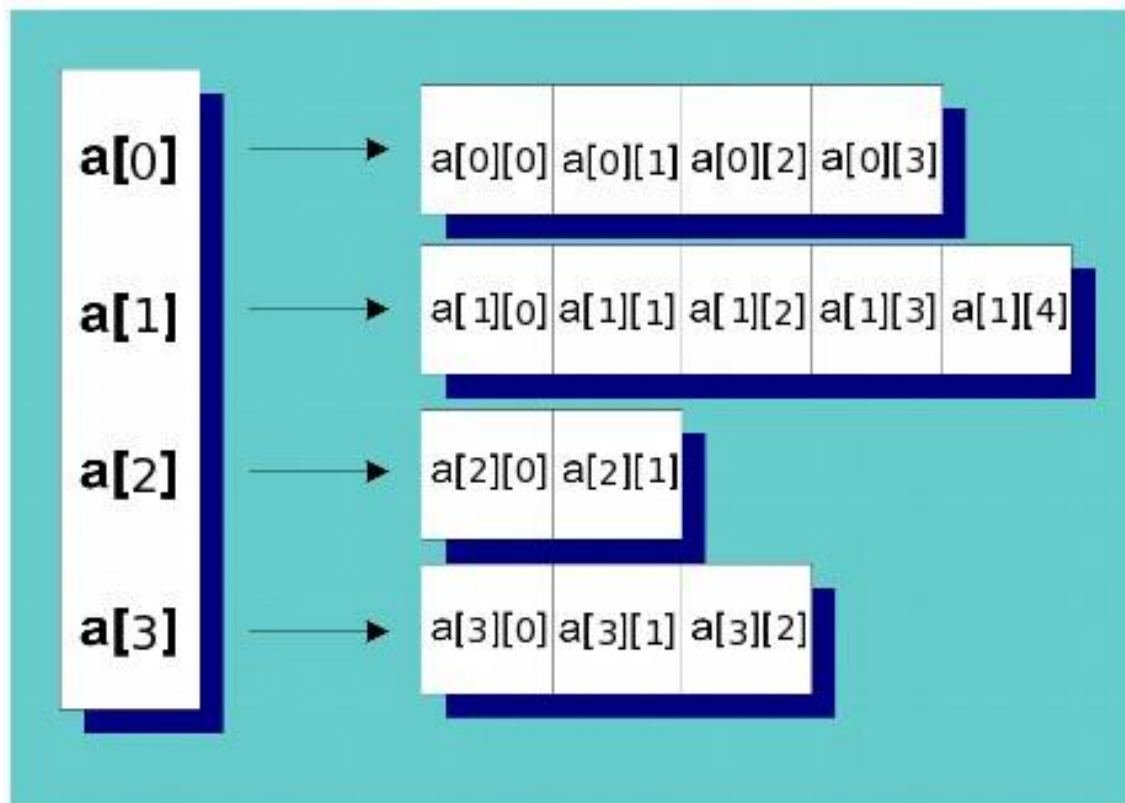
O declaración, construcción e inicializando directa de la matriz:

```
int[][] matriz = {{15, 20, 25, 30},  
                  {20, 30, 40, 50},  
                  {60, 65, 70, 80}};
```

Si solo hemos declarado y construido la matriz, para inicializarla utilizaremos bucles anidados, tantos como dimensiones tenga. En el caso de las bidimensionales, 2.

```
for (int i=0; i < matriz.length; i++){  
    for(int j=0; j < matriz[i].length; j++){  
        matriz[i][j] = i + j;  
    }  
}
```

Habr  ocasiones en las que no todos los arrays de la matriz tengan la misma longitud...



En estas situaciones primer definir mos el array que representa la primera dimensi n, y despu s ir mos asociando los arrays de la segunda dimensi n, cada uno en su posici n. Como es habitual, tenemos distintas opciones:

Creando el array base, después los arrays de las filas y finalmente asignando cada fila a su posición...

```
int[][] matriz = new int[4][];  
int[] matrizFila0 = new int[4];  
int[] matrizFila1 = new int[5];  
int[] matrizFila2 = new int[2];  
int[] matrizFila3 = new int[3];  
  
matriz[0] = matrizFila0;  
matriz[1] = matrizFila1;  
matriz[2] = matrizFila2;  
matriz[3] = matrizFila3;
```

Creando directamente un array en cada posición...

```
int[][] matriz2 = {new int[4],  
                  new int[5],  
                  new int[2],  
                  new int[3]};
```

Creando y asignando directamente...

```
int[][] matriz3 = {{00, 01, 02, 03},  
                  {10, 11, 12, 13, 14},  
                  {20, 21},  
                  {30, 31, 32}};
```

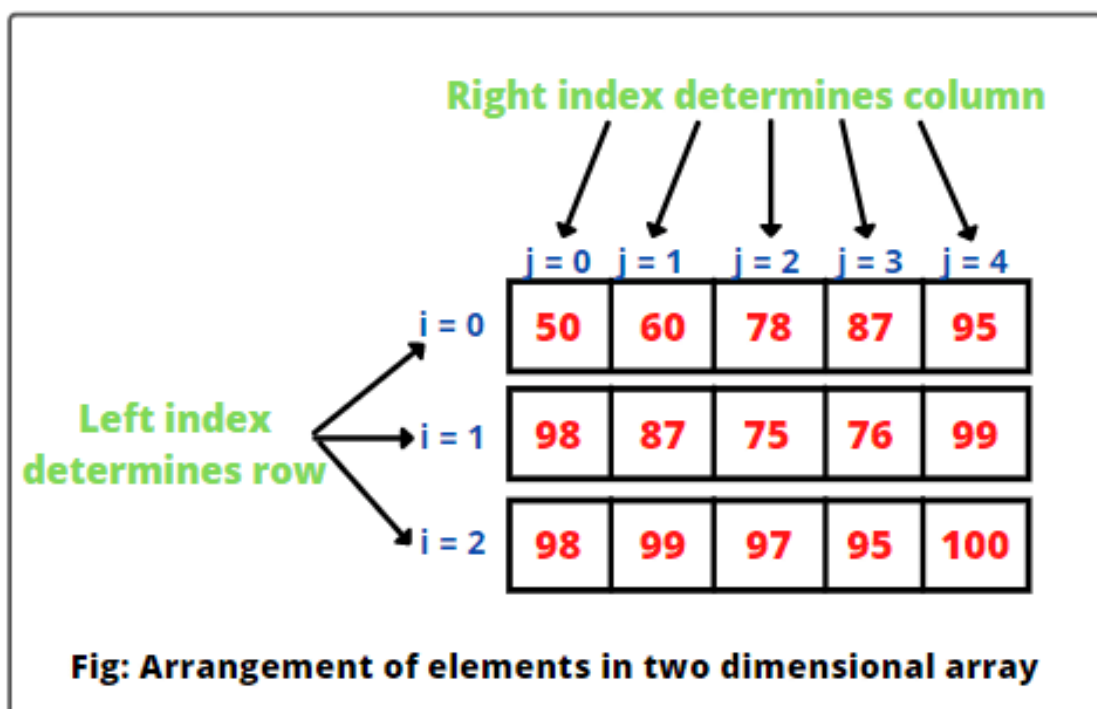
En el caso de no utilizar última opción, la inicialización la haríamos de la misma forma que hemos visto anteriormente:

```
for (int i=0; i < matriz.length; i++){  
    for(int j=0; j < matriz[i].length; j++){  
        matriz[i][j] = i + j;  
    }  
}
```

10.2. Acceso a los datos de un matriz

Como ya hemos visto en la inicialización mediante bucles, para acceder a uno de los elementos de una matriz lo haremos de forma similar a cómo lo hacíamos con un array unidimensional, pero añadiremos otro grupo de corchetes con la posición de la segunda dimensión.

	Col1	Col2	Col3	Col4	
Row1	Arr[0][0]	Arr[0][1]	Arr[0][2]	Arr[0][3]	
Row2	Arr[1][0]	Arr[1][1]	Arr[1][2]	Arr[1][3]	
Row3	Arr[2][0]	Arr[2][1]	Arr[2][2]	Arr[2][3]	
Row4	Arr[3][0]	Arr[3][1]	Arr[3][2]	Arr[3][3]	
⋮					



Para insertar un valor en una posición de una matriz...

```
matriz[2][3] = 95;
```

Para obtener el valor de una posición de una matriz...

```
int numero = matriz[2][3];
```

11. CADENAS DE CARACTERES. STRING.

11.1. Tipo primitivo char

Este tipo primitivo ya lo hemos visto anteriormente, y sabemos que los valores corresponden a códigos numéricos (code point) que son representados mediante letras, números, símbolos, caracteres especiales, etc... que nos encontramos en la codificación Unicode.

A la hora de seleccionar un carácter es posible usar su codificación Unicode o el propio carácter si es posible escribirlo mediante teclado.

```
char c;  
c = 'a'; //directamente mediante el teclado  
c = 97; //usando el code point en decimal  
c = '\u0061'; //o con el code point en hexadecimal
```

En realidad, para codificar cualquier code point de Unicode necesitamos 3 bytes, mientras que el tipo char solo utiliza 2 bytes, de tal forma que no todos los code points pueden representarse en este tipo primitivo, concretamente los que tengan un valor inferior o igual a 65535 (\uFFFF en hexadecimal). En el caso de querer almacenar el code point de un carácter Unicode superior a este valor debemos utilizar una variable de int que se almacena en 4 bytes.

Algunos caracteres especiales ya los vimos en la unidad de entrada/salida,... son las secuencias de escape, que tienen un significado especial a la hora de representarse por pantalla.

Carácter	Nombre
\b	Borrado a la izquierda
\n	Nueva línea
\r	Retorno de carro
\t	Tabulador
\f	Nueva página
\'	Comilla simple
\"	Comilla doble
\\	Barra invertida

11.1.1. Conversión $\text{char} \leftrightarrow \text{int}$

Como hemos visto antes los code points no son más que números enteros, por tanto, la conversión del tipo char a int es prácticamente directa.

Aprovechando los mecanismos de conversión podemos realizar asignaciones de la forma:

```
int e = 'a'; //asigna un carácter a una variable int
System.out.println(e); // muestra 97
e = '\u0061'; //asigna un carácter a una variable int
System.out.println(e); // muestra 97
char c = 97; //asigna un entero a una variable char
System.out.println(c); // muestra 'a'
```

También podemos forzar la conversión por medio de un cast:

```
char c = 'a';
System.out.println((int) c); //muestra 97
int e = 97;
System.out.println((char) e); //muestra 'a'
```

Se pueden realizar asignaciones de variables del tipo int a char con un cast (realizando una conversión por estrechamiento):

```
int e = 97;
char c = (char) e; //c vale 'a'
```


O también se pueden realizar asignaciones del tipo char a int de forma implícita (realizando una conversión por ensanchamiento):

```
char c = 'a';  
int e = c; //e vale 97
```

Tanta es la estrecha relación que hay entre el tipo char y el int, que se nos permite realizar operaciones aritméticas con sus valores.

```
char c = 'a' + 1; //c tendrá valor 98, es decir, el caracter 'b'
```

11.2. Clase Character

La clase Character es un tipo de clase a la que se le denomina wrapper (envoltorio), las cuales nos proporcionan una extensión de un carácter primitivo, char en este caso, añadiendo nuevas funcionalidades para operar con él.

11.2.1. Clasificación de caracteres

Para la clase Character existen 4 tipos de caracteres:

- **Dígitos:** formado por los números del 0 al 9.
- **Letras:** formado por todos los elementos del alfabeto tanto minúsculas como mayúsculas.
- **Caracteres blancos:** como el espacio o la tabulación, entre otros.
- **Otros caracteres:** signos de puntuación, matemáticos, ...

Esto nos sirve porque la clase Character nos proporciona una serie de métodos que nos permiten determinar de qué tipo es un carácter.

- **boolean isDigit(char c):** devuelve true si c es un dígito, false en caso contrario.
- **boolean isLetter(char c):** devuelve true si c es una letra, false en caso contrario.

- **boolean isLetterorDigit(char c):** devuelve true si c es una letra o un dígito (carácter alfanumérico), false en caso contrario.
- **boolean isLowerCase(char c):** devuelve true si c es una letra en minúscula, false en caso contrario.
- **boolean isUpperCase(char c):** devuelve true si c es una letra en mayúscula, false en caso contrario.
- **boolean isSpaceChar(char c):** devuelve true si c es específicamente un espacio en blanco, false en caso contrario.
- **boolean isWhiteSpace(char c):** devuelve true si c es un carácter blanco, false en caso contrario. Estos caracteres son:
 - Espacio en blanco
 - Retorno de carro
 - Nueva línea
 - Tabulador
 - Otros (están en desuso)

Así se utilizarían:

```
if(Character.isDigit(c)){  
    ...  
}  
  
if(Character.isLetter(c)){  
    ...  
}  
  
if(Character.isLetterorDigit(c)){  
    ...  
}  
  
if(Character.isLowerCase(c)){  
    ...  
}  
  
if(Character.isUpperCase(c)){  
    ...  
}
```

```
if(Character.isSpaceChar(c)){  
    ...  
}  
  
if(Character.isWhiteSpace(c)){  
    ...  
}
```

11.2.2. Conversión entre caracteres

Dentro del grupo de caracteres de tipo letra podremos hacer dos tipos de conversiones mediante los siguientes métodos:

- `char toLowerCase(char c)`: convierte `c` en la misma letra pero en minúscula.

```
char c = 'A';  
c = Character.toLowerCase(c);  
//Ahora c es 'a'
```

- `char toUpperCase(char c)`: convierte `c` en la misma letra pero en minúscula.

```
char c = 'a';  
c = Character.toUpperCase(c);  
//Ahora c es 'A'
```

También tenemos la opción de convertir un carácter que corresponda a un dígito al tipo `int` mediante el método **`int getNumericValue(char carácter)`**.

```
char c = '9';  
int num = Character.getNumericValue(c); //num es 9
```

11.2. Cadenas de caracteres



Aunque llevamos mucho tiempo utilizando la clase `String` (cadena de caracteres) apenas hemos aprovechado su potencial. Como ya sabemos los `String` son conjuntos secuenciales de caracteres y como tales, vamos a poder manipularlos según nuestros intereses. A continuación veremos una gran variedad de métodos para poder trabajar con `Strings`.

11.2.1. Valores de otros tipos

Vamos a poder convertir valores de otros tipos en `Strings` y a la inversa `Strings` a valores de otros tipos.

Para la primera opción contamos con el método **`valueOf()`** de la clase `String`:

```
String cad;  
cad = String.valueOf(1234); //cad ahora es "1234"  
cad = String.valueOf(-12.34); //cad ahora es "-12.34"  
cad = String.valueOf('C'); //cad ahora es "C"  
cad = String.valueOf(true); //cad ahora es "true"
```

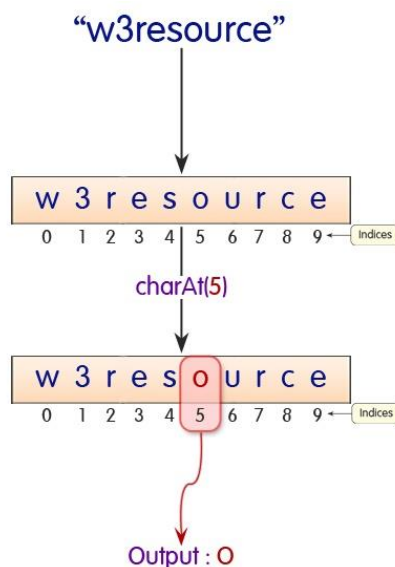
En el caso contrario disponemos del mismo método **`valueOf()`** de las clases wrapper de otros tipos de datos, como `Integer`, `Double`, `Float` y `Boolean`.

```
String cad;  
cad = "2";  
int numInt = Integer.valueOf(cad);  
cad = "2.5";  
double numDouble = Double.valueOf(cad);  
cad = "5.2";  
float numFloat = Float.valueOf(cad);
```

```
cad = "true";  
boolean bool = Boolean.valueOf(cad);
```

En el caso del tipo char vamos a hacer uso una función propia de los Strings denominada `charAt` que nos permitirá obtener el carácter que ocupa una posición indicada en un String.

```
String letra = "H";  
c = letra.charAt(0); //c ahora es 'H'  
  
String palabra = "Hola";  
c = letra.charAt(2); // c ahora es 'l'
```



11.2.2. Comparación de Strings

Para compara dos strings disponemos de varios métodos distintos:

- **boolean equals(String cadena):** devuelve true si la cadena invocante y la que se para por parámetro son exactamente iguales, o false en caso contrario.

```
String myStr1 = "Hello";  
String myStr2 = "Hello";  
String myStr3 = "Another String";  
System.out.println(myStr1.equals(myStr2)); // Returns true because  
they are equal  
System.out.println(myStr1.equals(myStr3)); // false
```

- **boolean equalsIgnoreCase(String cadena):** similar al método equals, pero sin distinguir entre mayúsculas y minúsculas.
- **boolean regionMatches(int inicio, String otraCad, int inicioOtra, int longitud):** compara dos fragmentos de cadenas: el primero corresponde a la cadena invocante y comienza en el carácter con índice inicio; y el segundo corresponde a la cadena otraCad y comienza en el carácter con índice inicioOtra. Ambos fragmentos tendrán la longitud indicada. El método devuelve true o false para indicar si las regiones coinciden.

```
String Str1 = new String("Welcome to Tutorialspoint.com");
String Str2 = new String("Tutorials");
String Str3 = new String("TUTORIALS");

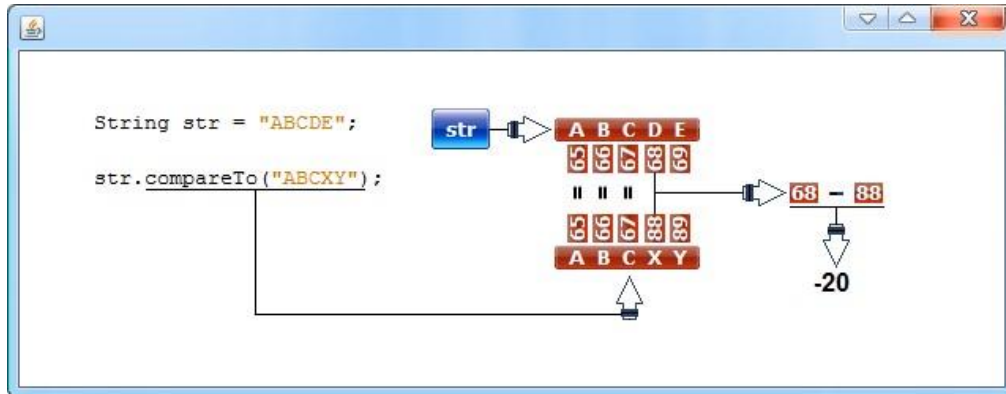
System.out.print("Return Value :" );
System.out.println(Str1.regionMatches(11, Str2, 0, 9));

System.out.print("Return Value :" );
System.out.println(Str1.regionMatches(11, Str3, 0, 9));

Return Value :true
Return Value :false
```

- **boolean regionMatches(boolean ignora, int inicio, String otraCad, int inicioOtra, int longitud):** hace lo mismo que el método anterior, pero si el parámetro ignora está a true, no distingue entre mayúsculas y minúsculas.
- **int compareTo(String cadena):** compara alfabéticamente la cadena que invoca el método y la que se pasa por parámetro, de tal forma que devolverá un entero cuyo valor determina el orden de las cadenas de la forma:
 - 0: si las cadenas son exactamente iguales
 - Número negativo: si la cadena invocante es menor alfabéticamente que la pasada por parámetro, es decir, va antes en orden alfabético.
 - Número positivo: si la cadena invocante es mayor alfabéticamente que la pasada por parámetro, es decir, va después en orden alfabético.

```
String myStr1 = "Hello";
String myStr2 = "Hello";
System.out.println(myStr1.compareTo(myStr2)); // Returns 0
because they are equal
```



- **int compareToIgnoreCase(String cadena):** funciona igual que compareTo, pero sin distinguir entre mayúsculas y minúsculas.

11.2.3. Concatenación



Conocemos concatenación como la unión de dos cadenas de caracteres, de tal forma que una queda detrás de la otra.

Para realizar esta operación utilizaremos los operadores "+" y "+=". El método concat() también se puede utilizar, pero no suele ser habitual.

```
String first = "Hello";
String second = "World";
String third = first + second;
```

```
System.out.println(third); //"HelloWorld"

// yet another way to concatenate strings
first += second;
System.out.println(first); //"HelloWorld"
```

```
String first = "Hello";
String second = "World";

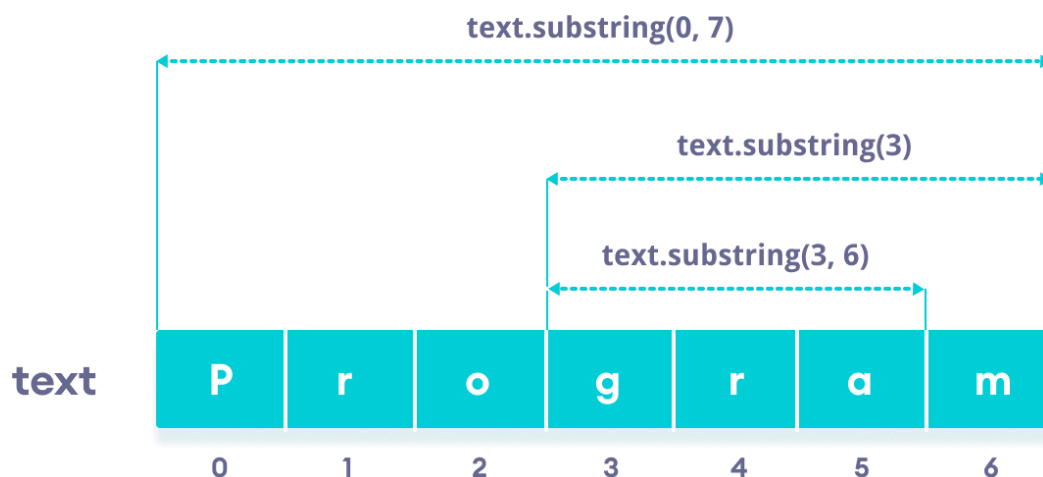
String third = first.concat(second);
System.out.println(third); //"HelloWorld"
```

11.2.4. Obtención de subcadenas

Otra posibilidad que tenemos con los Strings es extraer secciones de ellos, creando otros Strings. Para ello tenemos los siguientes métodos:

- **String substring(int inicio):** devuelve una subcadena formada desde la posición de inicio hasta el final de la cadena.
- **String substring(int inicio, int fin):** devuelve una subcadena formada desde la posición de inicio hasta la anterior posición fin indicadas.

```
String s1="javatpoint";
System.out.println(s1.substring(2,4));//returns va
System.out.println(s1.substring(2));//returns vatpoint
```



11.2.5. Longitud de una cadena

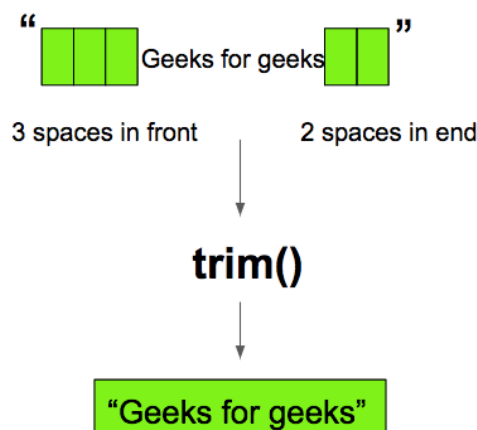
Podremos conocer la longitud de una cadena mediante el método **int length()**.

```
String s1="java";  
int longitud = s1.length(); //longitud es igual a 4
```

11.2.6. Eliminación de espacios vacíos

También será posible eliminar los espacios en blanco que tengamos en un String, tanto al comienzo como al final del mismo. Para ello utilizaremos el método **String trim()**.

```
String myStr = "    Hello World!    ";  
System.out.println(myStr); // "    Hello World!    "  
System.out.println(myStr.trim()); //"Hello World!"
```



11.2.7. Conversión de cadenas de caracteres

Disponemos de distintos métodos para modificar el contenido de una cadena de caracteres, como, por ejemplo:

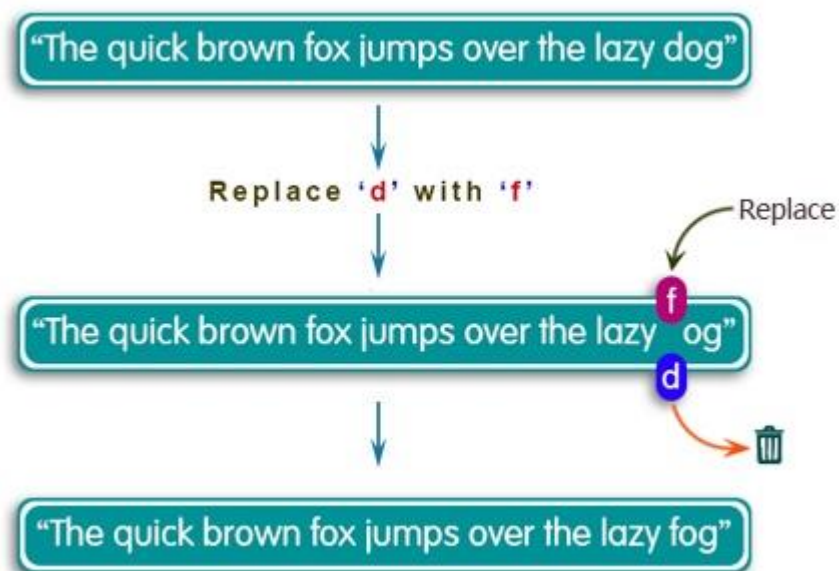
- **String toLowerCase()**: devuelve una copia de la cadena donde se han convertido las letras a minúsculas.
- **String toUpperCase()**: devuelve una copia de la cadena donde se han convertido las letras a mayúsculas.

```
String txt = "Hello World";  
System.out.println(txt.toUpperCase()); // "HELLO WORLD"
```

```
System.out.println(txt.toLowerCase()); // "hello world"
```

- **String replace(char original, char otro):** devuelve una copia de la cadena invocante donde se han sustituido todas las ocurrencias del carácter original por otro.
- **String replace(CharSequence original, CharSequence otro):** igual que el anterior, pero en lugar de sustituir una carácter por otro, sustituye una cadena de caracteres por otra.

```
String myStr = "Hello";  
  
System.out.println(myStr.replace('l', 'p')); // "Heppo"  
System.out.println(myStr.replace("llo", "re")); // "Here"
```

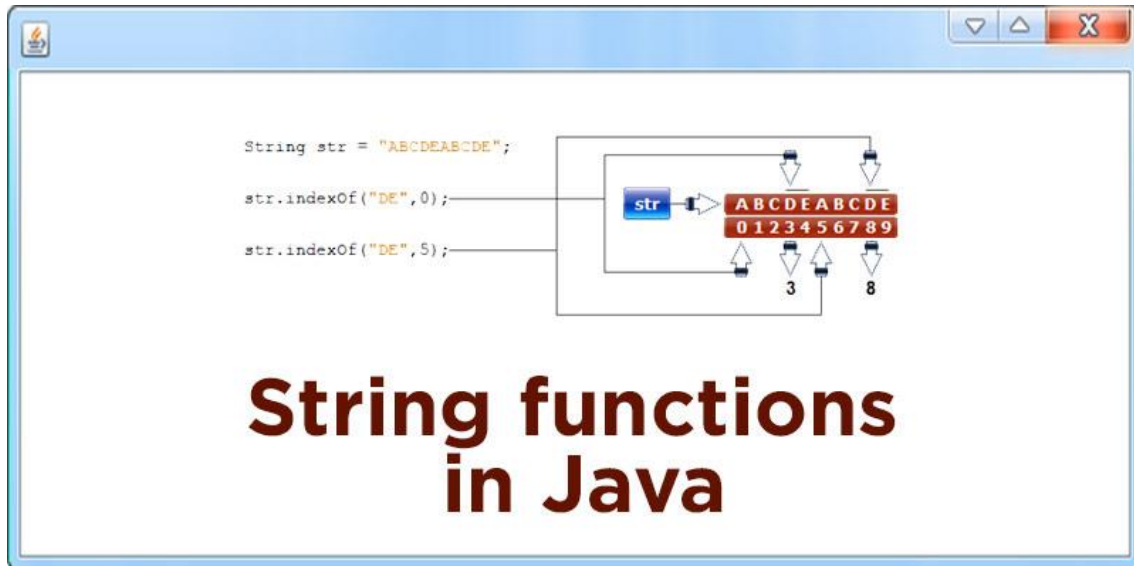


11.2.8. Búsqueda

Dentro de un String tendremos la posibilidad de localizar tanto caracteres como subcadenas de caracteres con los siguientes métodos:

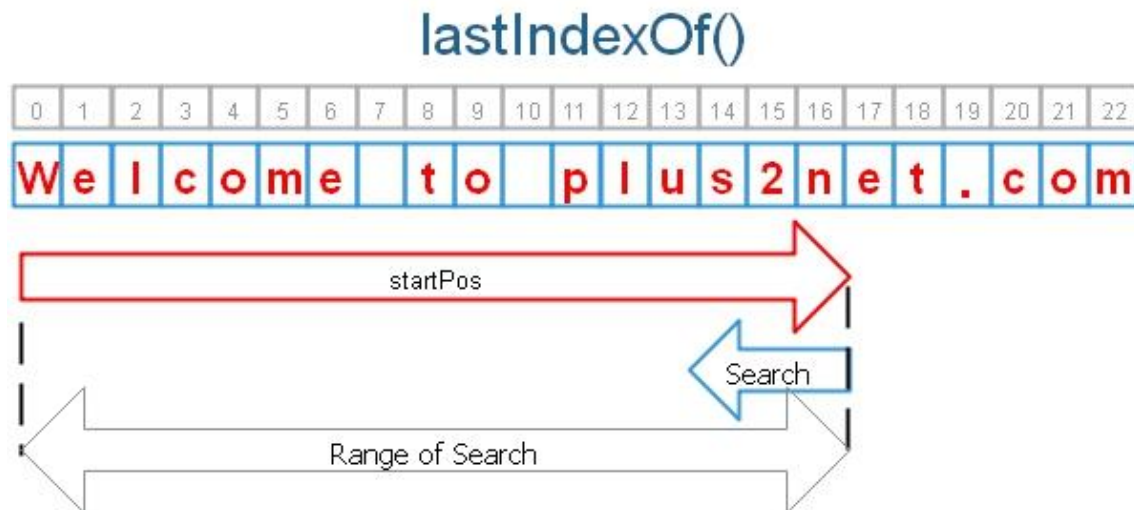
- **int indexOf(int c):** nos devuelve la posición de la primera aparición del carácter c en una cadena de caracteres. Si no lo encuentra devuelve -1.
- **int indexOf(String cadena):** igual que el anterior, pero busca una cadena de caracteres, en lugar de un único carácter.

- **int indexOf(int c, int inicio):** igual que la primera versión, pero comienza a buscar desde la posición de inicio.
- **int indexOf(String cadena, int inicio):** igual que la segunda versión, pero comienza a buscar desde la posición de inicio.



Los anteriores métodos buscaban la primera aparición de un carácter o una subcadena de caracteres. Las siguientes buscan la última aparición dentro de la cadena de caracteres.

- **int lastIndexOf(int c):** nos devuelve la posición de la última aparición del carácter c en una cadena de caracteres. Si no lo encuentra devuelve -1.
- **int lastIndexOf (String cadena):** igual que el anterior, pero busca una cadena de caracteres, en lugar de un único carácter.
- **int lastIndexOf (int c, int inicio):** igual que la primera versión, pero comienza a buscar desde la posición de inicio hacia el principio de la cadena de caracteres.
- **int lastIndexOf (String cadena, int inicio):** igual que la segunda versión, pero comienza a buscar desde la posición de inicio hacia el principio de la cadena de caracteres.



```
String myStr = "Hello planet earth, you are a great planet.";
System.out.println(myStr.lastIndexOf("planet")); //36
System.out.println(myStr.lastIndexOf("planet", 20)); //6
```

11.2.9. Comprobaciones

Tenemos la posibilidad de comprobar si una cadena de caracteres es vacía mediante el método **isEmpty()**.

```
String cadenaVacía = "";
String cadenaHola = "Hola";

if(cadenaVacía.isEmpty()){ //true
    ...
}

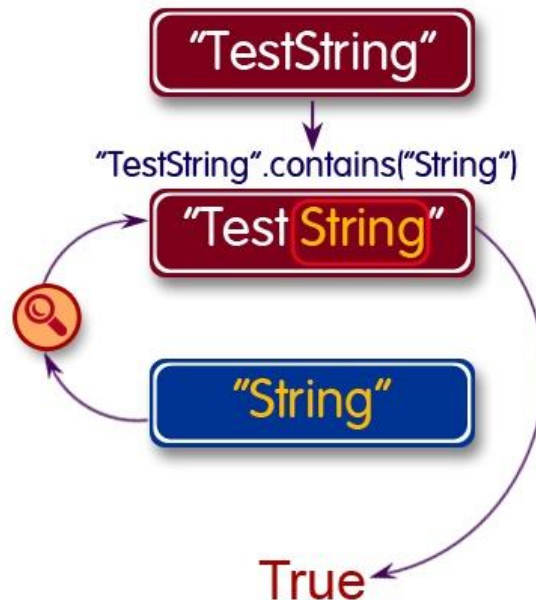
if(cadenaHola.isEmpty()){ //false
    ...
}
```

Por otra parte, también nos es posible conocer si una subcadena está contenida dentro de otra cadena de caracteres. Esto lo haremos a través del método **contains()**.

```
String cadena = "En un lugar de la Mancha";
String palabra = "lugar";

if(cadena.contains(palabra)){ //true
    ...
}
```

```
}  
  
if(cadena.contains("Hola")){ //false  
    ...  
}
```



Finalmente, podemos comprobar si una cadena de caracteres comienza o termina con una subcadena determinada. Para ello haremos uso de:

- **boolean startsWith(String prefijo):** comprueba si la cadena que invoca el método comienza con la subcadena prefijo.
- **boolean startsWith(String prefijo, int inicio):** hace lo mismo que la anterior pero empieza a buscar desde la posición de inicio.
- **boolean endsWith(String sufijo):** indica si la cadena termina con el sufijo que le pasamos por parámetro.

```
String cadena = "En un lugar de la Mancha";  
String palabra = "lugar";  
  
if(cadena.startsWitch(palabra)){ //false  
    ...  
}  
  
if(cadena.startsWith(palabra, 6)){ //true  
    ...  
}
```

```
}  
  
if(cadena.endsWith("Mancha")){ //true  
    ...  
}
```

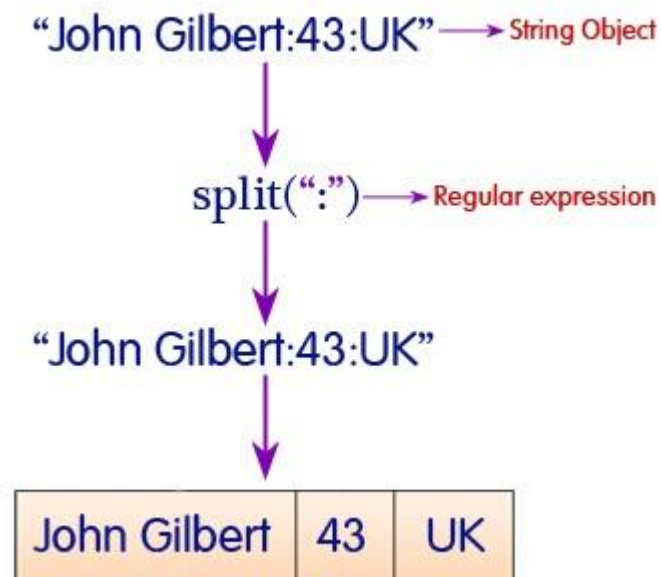


11.2.10. Separación en partes

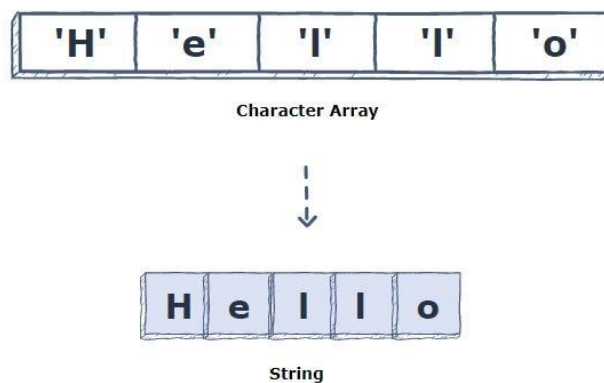
Otra funcionalidad muy útil con los Strings es la dividirlo en partes separadas por una subcadena. Por ejemplo " " o "," suelen ser de los más habituales, aunque es posible utilizar subcadenas de mayor tamaño. Estas subcadenas no podrán contener ninguno de los siguientes caracteres (., +, *, \$ o ?).

El método que utilizaremos para separar distintas partes será **String[] Split(String separador)**, que nos devolverá un array de Strings con las partes separadas y en el que habrán desaparecido todas las ocurrencias de la subcadena separador.

```
String cadena = "En un lugar de la Mancha";  
  
String[] cadenaDividida = cadena.split(" ");  
//cadenaDividida contiene {"En", "un", "lugar", "de", "la", "Mancha"}
```



11.3. CADENAS Y ARRAYS DE CARACTERES



Existe una innegable relación entre las cadenas de caracteres, clase `String`, y los arrays de caracteres `char[]`. Tal es así que nos va a ser posible convertir arrays de caracteres a `Strings`, ya sea de forma parcial o completa. Para ello usaremos el método `valueOf()` que vimos anteriormente.

- **`static String valueOf(char[] charArray)`**: devuelve un `String` con el contenido del array que se pasa por parámetro.
- **`static String valueOf(char[] charArray, int inicio, int longitud)`**: también devuelve un `String`, pero en este caso el contenido abarcará la longitud indicada a partir de la posición de inicio.

```
String cadena;  
  
char charArray[] = {'H', 'o', 'l', 'a'};  
cadena = String.valueOf(charArray);  
//cadena tiene el valor "Hola"  
  
charArray[] = {'a', 'b', 'c', 'd', 'e', 'f'};  
cadena = String.valueOf(charArray, 2, 3);  
//cadena tiene el valor "cde"
```

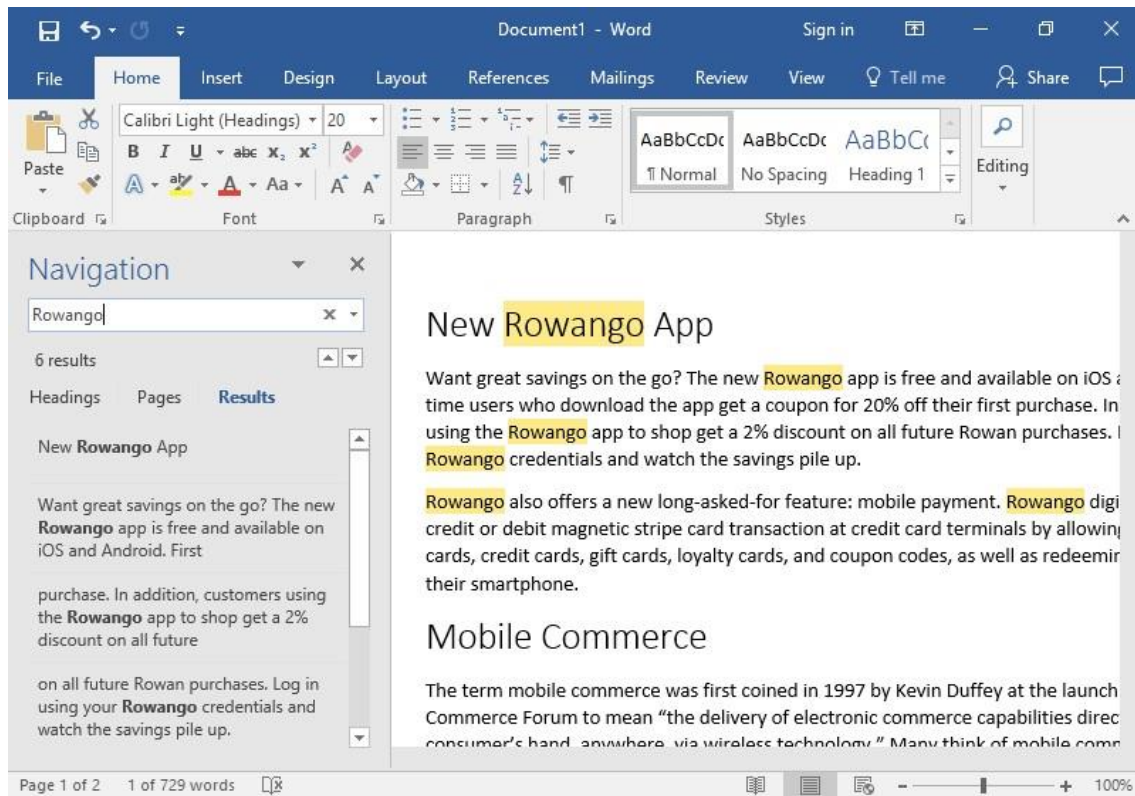
También podremos realizar la operación inversa, es decir, convertir un `String` en un array de caracteres mediante el método `char[] toCharArray()`.

```
char[] charArray;  
String cadena = "Hola mundo";  
  
charArray = cadena.toCharArray();  
//el valor de charArray es {'H','o','l','a',' ','m','u','n','d','o'}
```

12. EXPRESIONES REGULARES

Las expresiones regulares (comúnmente denominadas como *regex*) son secuencias de caracteres que conforman un patrón de búsqueda. Por lo general, estas búsquedas se realizan sobre cadenas de caracteres, es decir, sobre texto plano.

Actualmente las expresiones regulares tienen una gran utilidad puesto que se utilizan en una gran cantidad de ámbitos, desde el buscador de texto que incluyen las aplicaciones ofimáticas como Word o buscadores web, hasta la detección de expresiones clave en con correos electrónicos y mensajes de texto por parte de las fuerzas de seguridad, pasando por los validadores en la creación de cuentas de usuarios y contraseñas.



Las expresiones tienen una gran complejidad y su dominio requiere de un profundo conocimiento de la materia, es por ello que en esta unidad únicamente vamos a conocer cómo utilizarlas de una forma superficial.

Para hacer uso de las expresiones regulares debemos conocer los cuantificadores y metacaracteres con los que se construyen.

12.1. Cuantificadores para una expresión regular

Tenemos caracteres especiales que nos van a indicar el número de repeticiones de la expresión, la siguiente tabla muestra los caracteres:

Cuantificador	Descripción
n+	Encuentra cualquier string con al menos un «n»
n*	Encuentra cero o más ocurrencias de n
n?	Encuentra en el string la aparición de n cero o una vez
n{x}	Encuentra la secuencia de n tantas veces como indica x.
n{x,}	Encuentra una secuencia de n al menos tantas veces como indica n.

12.2. Metacaracteres en una expresión regular

Metacaracter	Descripción
	Símbolo para indicar OR.
.	Encuentra cualquier carácter
^	Sirve para hacer match al principio del string
\$	Hace match al final de un String
\d	Encuentra dígitos
\s	Busca un espacio
\b	Hace match al principio de una palabra.

12.3. Metacaracteres y ejemplos con expresiones regulares

Expresión regular	Descripción
.	Hace match con cualquier caracter
^regex	Encuentra cualquier expresión que coincida al principio de la línea.
regex\$	Encuentra la expresión que haga match al final de la línea.
[abc]	Establece la definición de la expresión, por ejemplo, la expresión escrita haría match con a, b o c.
[abc][vz]	Establece una definición en la que se hace match con a, b o c y a continuación va seguido por v o por z.
[^abc]	Cuando el símbolo ^ aparece al principio de una expresión después de [, negaría el patrón definido. Por ejemplo, el patrón anterior negaría el patrón, es decir, hace match para todo menos para la a, la b o la c.
[e-f]	Cuando hacemos uso de -, definimos rangos. Por ejemplo, en la expresión anterior buscamos hacer match de una letra entre la e y la f.
Y X	Establece un OR, encuentra la Y o la X.
HO	Encuentra HO (texto literal)
\$	Verifica si el final de una línea sigue.

12.4. Ejemplo simples de expresiones regulares

- Número entero de exactamente 6 dígitos: “\d{6}”
- Palabras en minúscula compuestas por 6 letras como máximo, con un espacio en blanco delante y otro detrás: “\b[a-z]{1,6}\b”
- Una línea compuesta por una única palabra compuesta por letras en mayúsculas y/o minúsculas: “^[a-zA-Z]+\$”
- Palabra que empieza por una letra mayúscula y el resto minúsculas: “[A-Z]{1}[a-z]+”
- Contraseña compuesta por entre 8 y 15 caracteres alfanuméricos: “[a-zA-Z0-9]{8,15}”

En las siguientes páginas web podéis encontrar información sobre cómo crear expresiones regulares, ejemplos de algunas de las más usadas y simuladores para probarlas:

- <https://www.freeformatter.com/java-regex-tester.html#before-output>
- <https://geekebrains.com/code-brains/expresiones-regulares>
- <https://ihateregex.io/playground>

12.5. Uso de expresiones regulares en Java

En Java utilizaremos dos clases específicas para aplicar esta herramienta tan potente: **Matcher** y **Pattern**.

En primer lugar, necesitaremos importarlas:

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;
```

Y a continuación crearemos la instancia de cada una de ellas:

```
String prueba = "123456";
Pattern pattern = Pattern.compile("\\d{6}");
//cuando tengamos que poner una barra invertidas será necesaria poner dos
juntas
Matcher matcher = pattern.matcher(prueba);
```

Finalmente utilizaremos uno de los métodos que nos proporciona la clase `Matcher` para comprobar la expresión regular con el texto:

- **`boolean find()`**: comprueba si existe al menos una coincidencia de la expresión en todo el texto proporcionado.
- **`boolean find(int ini)`**: comprueba si existe al menos una coincidencia de la expresión en el texto proporcionado a partir de la posición `ini`.
- **`boolean lookingAt()`**: comprueba si existe una coincidencia de la expresión regular justo al inicio del texto.
- **`boolean matches()`**: comprueba si el texto coincide exactamente con la expresión regular.

```
if(matcher.find()){
    System.out.println("Encontrado");
}
else{
    System.out.println("No encontrado");
}

//contar cuantas veces se ha encontrado coincidencia
contador = 0;
while(matcher.find()){
    contador ++;
}

if(matcher.lookingAt()){
    System.out.println("Encontrado");
}
else{
    System.out.println("No encontrado");
}

if(matcher.matches()){
    System.out.println("Encontrado");
}
else{
    System.out.println("No encontrado");
}
```